
date: 2026-05-27 source: C:\Users\50469\Desktop\python实验\课堂老师程序 status: 已整理

课堂老师程序对照：第六章函数

关联笔记： [\[\[25-function-basics/theory_25_function_basics|25-function-basics\]\]](#) · [\[\[11-closure-decorator/theory_11_closure_decorator|11-closure-decorator\]\]](#) · [\[\[16-python-gaps/theory_16_python_gaps|16-python-gaps\]\]](#) · [\[\[10-fastapi/theory_10_fastapi|10-fastapi\]\]](#) · [\[\[16-python-gaps/笔记_高阶函数\]\]](#)

如果你只有时间读三句话

1. **老师这批代码在讲什么：** Python 函数的核心机制——定义、传参、返回值、作用域、嵌套、装饰器。这是你当初看 FastAPI 一头雾水的底层原因。你的 [\[\[25-function-basics/theory_25_function_basics|25-function-basics\]\]](#) 是同一批知识的更详细版本。
 2. **这和你什么关系：** FastAPI 的 `@app.get` 本质就是装饰器 ([\[\[11-closure-decorator/theory_11_closure_decorator|11-closure-decorator\]\]](#) 已经讲过) , `def create_note(note: NoteCreate)` 本质就是函数定义, `Depends()` 本质就是框架帮你传参。不懂函数机制, FastAPI ([\[\[10-fastapi/theory_10_fastapi|10-fastapi\]\]](#)) 就是空中楼阁。
 3. **怎么用这份笔记：** 按顺序看, 每个概念分四步——看老师代码 → 看解释 → 看 FastAPI 映射 → 关文件自己说一遍。
-

一、函数定义和调用

老师代码

```
def fn():  
    pass  
  
def add():
```

```
result = 11 + 22
print(result)

def add_modify(a, b):
    result = a + b
    return result

# 函数是对象，可以赋值
x = add
x()

# 调用有参函数
print(add_modify(10, 20))
```

它在说什么

现象	含义
<code>def fn(): pass</code>	定义一个什么都不做的函数
<code>add()</code> 输出 33	函数定义后 不会自动执行 ，必须用 <code>()</code> 调用
<code>x = add; x()</code>	函数是 第一类对象 ——可以赋值给变量
<code>add_modify(10, 20)</code>	带参数的函数：调用时传入实际值

理解： 函数是"先定义，后使用"的代码块。定义时只是注册，调用时才执行。这个区分是所有后续概念的基础。详细展开见 [\[\[25-function-basics/theory_25_function_basics|25-function-basics\]\]](#)。

FastAPI 映射

对应 [\[\[10-fastapi/theory_10_fastapi|10-fastapi\]\]](#) 的路由注册、[\[\[10-fastapi/theory_10_fastapi#16-Depends: 让-FastAPI-帮你拿对象|Depends 依赖注入\]\]](#)。

```
# 这就是一个普通的函数定义
@app.post("/notes")
def create_note(note: NoteCreate, service: NoteServiceDep) -> NoteRead:
    return service.create_note(note)
```

```
# 你不手动调用它——FastAPI 替你调用
# 浏览器发 POST /notes → FastAPI 拿到请求 → 调用 create_note(...)
```

💡 为什么你不明白——因为你没见过"有人替你调用"这种模式

到目前为止你见过的所有函数调用都是 **你主动写代码去调**：

```
python result = add_modify(10, 20) # ← 你亲自调用的
```

但 FastAPI 的模式是：**你只管定义函数，框架在合适的时机替你调用它。**

对比	老师代码	FastAPI
调用者	你自己写 <code>add_modify(10, 20)</code>	FastAPI 框架自动调用
触发时机	代码执行到这行	收到 HTTP 请求时
参数来源	你手动传入	框架从 HTTP 请求提取
return 结果去哪	回到你的代码里	框架接管，序列化后发回浏览器

打个比方：

场景	类比
普通函数调用	你打电话给外卖店 → 你主动拨号 → 对方接听 → 你拿到结果
FastAPI 路由	你开了一家店装了电话 → 你告诉店员怎么接 → 电话响时店员自动接 → 按你教的回答

函数本身完全没变——都是 `def 函数名(参数): 处理逻辑 return 结果`。变的只是**调用者从你变成了框架**。

二、return——函数把结果交到哪里

老师代码

```
# return 把结果“扔回”调用处
def filter_sensitive_words(words):
    if "山寨" in words:
        new_words = words.replace("山寨", "*")
    return new_words

result = filter_sensitive_words("这个手机是山寨版的吧！")
print(result) # 这个手机是*版的吧！
```

核心： `return` 不是打印，是把值传递给调用者。没有 `return`，函数返回 `None`。

多个返回值本质是元组

```
def move(x, y, step):
    nx = x + step
    ny = y - step
    return nx, ny

# 解包接收
a, b = move(100, 100, 60) # a=160, b=40

# 也可以当元组收
result = move(100, 100, 60)
print(type(result)) # <class 'tuple'>
print(result)      # (160, 40)
```

关键理解： Python 没有“返回多个值”，`return nx, ny` 实际上是 `return (nx, ny)` ——一个元组。

FastAPI 映射

```
@app.post("/notes")
def create_note(note: NoteCreate, service: NoteServiceDep) -> NoteRead:
    return service.create_note(note)

# return 的值 → FastAPI → 自动转 JSON → 返回给前端
```

老师代码里 `return` 交给 `print`，FastAPI 里 `return` 交给框架序列化。本质都是把值传到函数外面。

💡

return 之后发生了什么——FastAPI 内部帮你做了 3 件事

步骤	发生了什么	代码层面
第 1 步：拿到返回值	<code>service.create_note(note)</code> 返回一个 Python 对象，比如 <code>NoteRead(id=1, content="你好", ...)</code>	对象在内存中
第 2 步：序列化	FastAPI 调 Pydantic 的 <code>.model_dump()</code> 把对象转字典，再转成 JSON 字符串	<code>{"id":1, "content":"你好"}</code> → <code>'{"id":1,...}'</code>
第 3 步：发回浏览器	包装成 HTTP 响应报文，Uvicorn 通过 TCP 网络发出去	<code>HTTP/1.1 200 OK</code> + JSON 体

对比一下就懂了：

场景	流程
老师代码	<code>return a + b</code> → <code>print(result)</code> ← 结果到控制台就结束了
FastAPI	<code>return obj</code> → FastAPI 序列化 → HTTP 响应 → 网络传输 → 浏览器看到 JSON

区别不是 `return` 变了，而是 **return 之后接的东西不同**。

三、参数传递——怎么把数据送进函数

3.1 位置参数

```
def get_max(a, b):
    if a > b:
        print(a, "是较大的值")
    else:
        print(b, "是较大的值")

get_max(8, 5)  # 8 是较大的值 —— a=8, b=5, 按顺序匹配
```

3.2 关键字参数

```
def fn(a, b):
    print(a, b)

fn(b=5, a=10)  # 10 5 —— 指定名字传参，顺序无所谓
```

💡 通俗版：位置参数是"按座位坐"，关键字参数是"喊名字"

方式	类比	代码
位置参数	老师喊"按学号入座"——1 号坐第 1 个位，顺序必须对	<code>fn(8, 5)</code>
关键字参数	"张三坐李四的位置也行，只要你喊名字"——报名字就能对上	<code>fn(b=5, a=10)</code>

场景	用位置参数	用关键字参数
参数很少，一看就懂	<code>add(1, 2)</code> ✓	没必要
参数很多，怕搞混顺序	容易传错 ✗	<code>create_user(name="张三", age=20, city="南昌")</code> ✓
只传其中几个参数	必须全传 ✗	<code>fn(a=10)</code> 只传 a ✓

一句话：位置参数省事但死板，关键字参数啰嗦但保险。

3.3 仅限位置参数 (/)

```
def func(a, b, /, c):
    print(a, b, c)

func(10, 20, 30)      # 正确
func(10, 20, c=30)    # 正确
# func(a=10, b=20, c=30)  # 错误! a,b 不能用关键字传
```

3.4 默认参数

```
def fn(a, b, c=100):
    print(a, b, c)
```

```
fn(10, 20)           # 10 20 100
fn(10, 20, c=30)     # 10 20 30
```

FastAPI 映射

对应 [[10-fastapi/theory_10_fastapi#6-路径参数：从-URL-路径里拿值|路径参数]] 和 [[10-fastapi/theory_10_fastapi#7-查询参数：从问号后面拿值|查询参数]] 两节。

```
@app.get("/notes/{note_id}")           # note_id 从 URL 路径来（位置隐射）
def get_note(note_id: int, q: str = None): # q 有默认值，可选参数
    ...

@app.get("/items")
def list_items(skip: int = 0, limit: int = 10): # 查询参数默认值
    ...
```

本质： 框架从 HTTP 请求中提取数据，按参数名/位置传给函数。你定义的参数就是框架填的槽。

四、`*args` 和 `**kwargs` ——参数打包

打包

```
# *args: 把多个位置参数打包成元组
def test(*args):
    print(type(args)) # <class 'tuple'>
    print(args)       # (61, 22, 13, 44, 55)

test(61, 22, 13, 44, 55)

# **kwargs: 把多个关键字参数打包成字典
def test(**kwargs):
    print(type(kwargs)) # <class 'dict'>
    print(kwargs)       # {'a': 11, 'b': 22, 'c': 33, 'd': 44, 'e': 55}

test(a=11, b=22, c=33, d=44, e=55)
```

解包

```
# *tuple: 把元组拆开按位置传
def test(a, b, c, d, e):
    print(a, b, c, d, e)

nums = (11, 22, 33, 44, 55)
test(*nums) # 等价于 test(11, 22, 33, 44, 55)

# **dict: 把字典拆开按关键字传
nums = {"a": 11, "b": 22, "c": 33, "d": 44, "e": 55}
test(**nums) # 等价于 test(a=11, b=22, c=33, d=44, e=55)
```

理解：打包和解包是对称的——

位置	<code>*</code> 的作用	<code>**</code> 的作用
定义形参时	<code>*args</code> ：多个位置参数 → 元组	<code>**kwargs</code> ：多个关键字参数 → 字典
调用实参时	<code>*tuple</code> ：元组 → 多个位置参数	<code>**dict</code> ：字典 → 多个关键字参数

详细讲解见 [\[\[16-python-gaps/theory_16_python_gaps#5-args-和-kwargs|args/*kwargs\]\]](#) 和 [\[\[16-python-gaps/笔记_高阶函数\]\]](#)。

混合传递顺序规则

```
def test(a, b, c=33, *args, **kwargs):
    print(a, b, c, args, kwargs)

test(1, 2)           # 1 2 33 () {}
test(1, 2, 3)        # 1 2 3 () {}
test(1, 2, 3, 4)     # 1 2 3 (4,) {}
test(1, 2, 3, 4, e=5) # 1 2 3 (4,) {'e': 5}
```

顺序铁律：普通参数 → 默认参数 → `*args` → `**kwargs`

FastAPI 映射

```
def new_function(*args, **kwargs):  
    result = old(*args, **kwargs)  
    return result
```

装饰器里 `*args, **kwargs` 起到**透明转发**的作用——包装函数不知道原函数签名，但用打包再解包就能原样转发。这是 FastAPI 装饰器能包装各种不同参数路由函数的原因，见 [[16-python-gaps/笔记_高阶函数]] 的 `run` 函数模式。

五、作用域——变量在哪里有效

局部变量

```
def test_one():  
    number = 10      # 局部变量：函数内部有效  
    print(number)  
  
test_one()           # 10  
# print(number)      # 报错！函数外访问不到
```

不同函数可以有同名变量，互不影响

```
def test_one():  
    number = 10  
    print(number)    # 10  
  
def test_two():  
    number = 20  
    print(number)    # 20
```

全局变量——只能读不能直接改

```
number = 10          # 全局变量
```

```
def test_one():
    print(number)    # 能读：10
    # number += 2    # 报错！不能直接改——Python 认为你在定义局部变量

test_one()
print(number)        # 10（没变）
```

global——声明要改全局变量

```
number = 10

def test_one():
    global number    # 声明：我要改的是全局那个 number
    number += 1
    print(number)    # 11

test_one()
print(number)        # 11（真的改了）
```

nonlocal——改外层函数的变量

```
def test(a):
    number = 10
    def test_in():
        nonlocal number    # 声明：我要改的是外层函数的 number，不是全局
        number += 20
        print(number, a)    # 30 15
    test_in()
    print(number)           # 30（被内层改了）

test(15)
```

理解： 变量查找顺序——局部 → 外层函数 → 全局 → 内置（LEGB 法则，见 [[16-python-gaps/theory_16_python_gaps#10-变量作用域legb-规则|LEGB 规则]]）。



`nonlocal` 可以连续用吗？—— 可以，它会一级一级往上链

你问的是这种场景：三层嵌套，每层都用 `nonlocal`，能不能一路改到最外层？

```
python def outer(): x = 1 def middle(): nonlocal x # middle 说: x 是 outer 的 x x += 10 def inner(): nonlocal x # inner 说: x 是 middle 的 x (而 middle 的 x 就是 outer 的 x) x += 100 inner() print(f"middle 里 x = {x}") # 111 middle() print(f"outer 里 x = {x}") # 111

outer() ``
```

执行顺序	x 的值	谁改的
初始	1	outer 定义
middle() 中 x += 10	11	middle 的 nonlocal x 指向 outer
inner() 中 x += 100	111	inner 的 nonlocal x 指向 middle (而 middle 的 x 就是 outer 的 x)

机制： nonlocal 不是"跳到指定层"，而是"找最近的外层"。如果最近的外层本身也是 nonlocal ，它就顺着链条往上走，最终都指向同一个变量。

不过实际代码很少写三层 nonlocal 连环套——两层（闭包/装饰器）已经够用了。

关键字	作用域	用于
无	局部 / 全局	大多数情况
global	在函数内修改全局变量	配置、计数器
nonlocal	在内层函数修改外层函数的变量	闭包、装饰器

global 和 nonlocal 就是告诉 Python"跳过局部这个层级，直接去更外层找"。

六、嵌套函数和闭包

```
def add_modify(a, b):
    result = a + b
    print(result)
```

```
def test():                # 内部定义函数
    print("我是内层函数")

test()                     # 外层调用内层

add_modify(10, 20)         # 30 \n 我是内层函数
# 外部无法调用 test()——它只在 add_modify 内部可见
```

关键理解： 内层函数相当于外层函数的"私有工具"，外部代码看不到它。这是信息隐藏的一种方式。这是[[11-closure-decorator/theory_11_closure_decorator#二、闭包：函数记住了它出生时的环境|闭包]]的基础。

FastAPI 联系： 装饰器的 `new_function` 就是嵌套在 `begin_end` 内部的，装饰器的本质就是： 1. 外层函数接收原函数 2. 内层函数包装原函数（加额外逻辑） 3. 返回内层函数替换原函数

七、装饰器——函数之上的函数

老师代码

```
import time

def begin_end(old):
    """给原函数加开始/结束/耗时统计"""
    def new_function(*args, **kwargs):
        print('开始执行~~~~~')
        time_begin = time.time()

        result = old(*args, **kwargs) # 调用原函数

        time_end = time.time()
        print('执行结束~~~~~')
        ms = (time_end - time_begin) * 1000
        print(f"{old.__name__} 耗时 {ms:.2f} 毫秒")
        return result
    return new_function
```

```
# 用法一：手动装饰
def sub(a, b):
    time.sleep(1)
    return a - b

f2 = begin_end(sub)
print(f2(10, 20))  # sub 被加了前后打印和耗时

# 用法二：@语法糖
@begin_end
def add(a, b):
    time.sleep(2)
    return a + b

result = add(3, 2)  # 等价于 add = begin_end(add)
print(result)
```

装饰器拆解

```
@begin_end
def add(a, b): ...

↓ 等价于

add = begin_end(add)
```

`@begin_end` 不是"声明", 是在函数定义后立即执行了一个函数调用。

执行过程：

```
add = begin_end(add)
    → begin_end 接收 add 函数对象
    → 内部定义 new_function (闭包, 记住 old=add)
    → 返回 new_function
    → add 被替换为 new_function
```

调用 `add(3, 2)` 时：

- 实际调用的是 `new_function`
- 打印 "开始执行"

- 调用原来的 `add(3, 2)`
- 打印 “执行结束”
- 返回结果

FastAPI 映射

```
@app.get("/notes")
def list_notes(): ...

# 等价于
# 1. app.get("/notes") 先执行，返回一个装饰器
# 2. 装饰器接收 list_notes
# 3. 把 list_notes 注册到路由表中
# 4. list_notes 变成路由处理函数
```

区别只是老师写的 `begin_end` 接收函数，FastAPI 的 `app.get("/notes")` 先接收路径参数再返回装饰器。多了一层参数，本质一样。详细见 [[11-closure-decorator/theory_11_closure_decorator#四、带参数的装饰器：@app-get("/path")]]。

理解：装饰器 = 高阶函数（参数是函数，返回值也是函数）。它是 Python 让函数作为第一类对象这个特性的集大成者。

八、递归函数

```
# 求 n!
# 边界条件: n == 1 → 返回 1
# 递归公式: n > 1 → n * func(n-1)

def func(num):
    if num == 1:
        return 1
    else:
        return num * func(num - 1)

num = int(input("请输入一个整数: "))
```

```
result = func(num)

print(f"{num}! = {d}" % result)
```

递归两要素： 1. **边界条件：** 什么时候停止 (`n == 1`) 2. **递归公式：** 问题缩小一圈 (`n → n-1`)

执行 `func(4)` 的展开过程：

```
func(4) → 4 * func(3)
        → 3 * func(2)
            → 2 * func(1)
                → 1
            → 2 * 1 = 2
        → 3 * 2 = 6
    → 4 * 6 = 24
```

九、匿名函数 lambda

```
# lambda 参数列表 : 表达式

temp = lambda a, b: a + b
print(type(temp)) # <class 'function'>
print(temp(10, 20)) # 30

# 相当于
def temp(a, b):
    return a + b
print(temp(10, 20)) # 30
```

lambda 限制： 只能写一行表达式，不能写语句。适合简单回调、排序 key、Pandas apply 等场景。你的 [\[\[25-function-basics/theory_25_function_basics|25-function-basics\]\]](#) 和 [\[\[04-listcomp-class/theory_04_listcomp_lambda_class|04-listcomp-class\]\]](#) 笔记中都用过它。

十、知识图谱——函数专题全景

```

└─ 函数定义 (def)
  └─ 函数是一等公民 (可赋值、可传参)
└─ 返回值 (return)
  └─ 单个值
  └─ 多个值 → 本质是元组

└─ 函数基础 ───┘
  └─ 参数传递
    └─ 位置参数
    └─ 关键字参数
    └─ 仅限位置参数 (/)
    └─ 默认参数
    └─ 打包 *args / **kwargs
    └─ 解包 *tuple / **dict
    └─ 混合传递顺序规则
  └─ 作用域
    └─ 局部变量
    └─ 全局变量 (只读)
    └─ global (声明修改全局)
    └─ nonlocal (修改外层函数的变量)

Python 函数 进阶机制 ───┘
  └─ 嵌套函数 (内层函数是外层私有工具)
  └─ 递归 (边界 + 递推公式)
  └─ 匿名函数 lambda (一行表达式)

└─ 核心应用 ───┘ 装饰器
  └─ 本质: 高阶函数 (收函数 → 返回函数)
  └─ @语法糖: @foo ≡ func = foo(func)
  └─ *args, **kwargs 透明转发
  └─ 闭包: 内层函数记住外层变量

└─ FastAPI ───┘
  └─ @app.get("/path") → 带参数的装饰器
  └─ def create_note(...) → 路由函数
  └─ Depends() → 框架帮你传参
  └─ return → 交给框架序列化

```

十一、自检问题

学完之后，每道题能脱口而出才算真的懂了：

基础部分：

- 1. `def add(a, b): return a + b` 定义后，`add` 本身是什么类型？`add(1, 2)` 是什么？
- 2. `return` 和 `print` 的根本区别是什么？
- 3. 为什么 Python 可以 `a, b = func()` 接收两个返回值？底层发生了什么？
- 4. `def fn(a, b, /, c)` 中 `/` 的作用是什么？
- 5. `*args` 得到什么类型？`**kwargs` 得到什么类型？

进阶部分：

- 1. `test(*nums)` 是在做什么？`test(**nums)` 呢？用一句话说清楚。
- 2. 为什么函数内可以读全局变量，但不能直接改？
- 3. `global` 和 `nonlocal` 的区别一句话说清。
- 4. 装饰器 `@begin_end` 等价于哪一行普通代码？

FastAPI 连接部分：

- 1. `@app.get("/notes")` 和普通装饰器有什么区别和联系？
- 2. FastAPI 里 `return` 的值去了哪里，和老师代码有什么区别？
- 3. 为什么装饰器里要用 `*args, **kwargs` 来调用原函数？
- 4. FastAPI 的路径参数 (`{id}`)、查询参数 (`?skip=0`)、请求体 (`POST body`) 分别对应函数参数的哪种传递方式？

附录：最快消化路线（40 分钟版）

按顺序跑，每个文件跑完说一句"这个文件教会了我什么"：

时间	文件	一句话目标
5 分钟	<code>函数的定义和调用.py</code> + <code>函数的一个返回值.py</code>	看懂函数定义→调用→return 的完整链路

时间	文件	一句话目标
5 分钟	函数的多个返回值.py	理解"多个返回值"本质是元组
5 分钟	位置参数、关键字参数、默认参数	三种传参方式
5 分钟	打包1+2 + 解包1+2	掌握 * / ** 在定义和调用时的对称用法
5 分钟	嵌套函数 + nonlocal	理解外层/内层函数关系
10 分钟	装饰器.py	最核心：理解装饰器本质 = 高阶函数
5 分钟	回头看 notes-api 的 @app.post 和 def create_note	验证你是否真懂了